Bean vs Component

- @Bean
- @Primary
- @Qualifier
- @Bean(autowireCandidate = false)
- BeanFactoryPostProcessor , BeanPostProcessor, @Bean(initMethod = “”), initMethod in Spring
- @PostConstruct
- @PreDestroy
- Destroy method in spring
- afterPropertiesSet()
- ShutDown Hook, @Profile, @ConditionalOnProperty

<u>Transaction and Spring JPA</u>

@Transactional, @EnableTransactionManagement, Platform Transactional Manager, Entity Manager Factory, Internal Working of Transaction along with AOP, Isolation, Persistence Context, Enum, @Enumerated , Projection in JPQL using Spring JPA , nativeQuery in JPQL using Hibernate in JPA, Pagination, Pageable , Page, PageRequest , Cascading , Types of Cascades, Data Domain vs JPA Domain, orphanRemoval, N + 1 problem

```
public interface PatientRepository extends JpaRepository<Patient, Long> {  4 usages
    List<Patient> findByNameContainingOrderByIdDesc(String query);  no usages

    @Query("SELECT p FROM Patient p where p.bloodGroup = ?1")  no usages
    List<Patient> findByBloodGroup(@Param("bloodGroup") BloodGroupType bloodGroup);

    @Query("select p from Patient p where p.birthDate > :birthDate")  no usages
    List<Patient> findByBornAfterDate(@Param("birthDate") LocalDate birthDate);

    @Query("select new com.codingshuttle.youtube.hospitalManagement.dto.BloodGroupCountResponseEntity(p.bloodGroup," +  no usages
            " Count(p)) from Patient p group by p.bloodGroup")
//    List<Object[]> countEachBloodGroupType();
    List<BloodGroupCountResponseEntity> countEachBloodGroupType();

    @Query(value = "select * from patient", nativeQuery = true)  no usages
    List<Patient> findAllPatients();

    @Transactional  1 usage
    @Modifying
    @Query("UPDATE Patient p SET p.name = :name where p.id = :id")
    int updateNameWithId(@Param("name") String name, @Param("id") Long id);
```

```java
public interface PatientRepository extends JpaRepository<Patient, Long> {  8 usages
    @Query(value = "select * from patient", nativeQuery = true)  1 usage
    Page<Patient> findAllPatients(Pageable pageable);

    @Transactional  no usages
    @Modifying
    @Query("UPDATE Patient p SET p.name = :name where p.id = :id")
    int updateNameWithId(@Param("name") String name, @Param("id") Long id);


//    @Query("SELECT p FROM Patient p LEFT JOIN FETCH p.appointments a LEFT JOIN FETCH a.doctor")
    @Query("SELECT p FROM Patient p LEFT JOIN FETCH p.appointments")  1 usage
    List<Patient> findAllPatientWithAppointment();

}
```

Isolation Level	Dirty Read Possible	Non-Repeatable Read Possible	Phantom Read Possible
READ_UNCOMMITTED	Yes	Yes	Yes
READ_COMMITTED	No	Yes	Yes
REPEATABLE_COMMITTED	No	No	Yes
SERIALIZABLE	No	No	No

Default isolation level, depends upon the DB which we are using.
Like Most relational Databases uses READ_COMMITTED as default isolation, but again it depends upon DB to DB.

ISOLATION LEVEL	Locking Strategy
Read Uncommitted	**Read** : No Lock acquired **Write**: No Lock acquired
Read Committed	**Read**: Shared Lock acquired and Released as soon as Read is done **Write**: Exclusive Lock acquired and keep till the end of the transaction
Repeatable Read	**Read**: Shared Lock acquired and Released only at the end of the Transaction **Write**: Exclusive Lock acquired and Released only at the end of the Transaction
Serializable	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the Transaction.

Isolation Level	Dirty Read Possible	Non-Repeatable Read Possible	Phantom Read Possible
READ_UNCOMMITTED	Yes	Yes	Yes
READ_COMMITTED	No	Yes	Yes
REPEATABLE_COMMITTED	No	No	Yes
SERIALIZABLE	No	No	No

ISOLATION LEVEL	Locking Strategy
Read Uncommitted	**Read** : No Lock acquired **Write**: No Lock acquired
Read Committed	**Read**: Shared Lock acquired and Released as soon as Read is done **Write**: Exclusive Lock acquired and keep till the end of the transaction
Repeatable Read	**Read**: Shared Lock acquired and Released only at the end of the Transaction **Write**: Exclusive Lock acquired and Released only at the end of the Transaction
Serializable	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the Transaction.

Propagation

Different types of propagation - @Transactional (propagation = Propagation.REQUIRED) etc

AOP

@Aspect, Execution() , (*) wildcard, (.)wildcard, Within() , @within, @annotation, @Before("args(String,int)"), @args, target , combining pointcuts using &&(boolean and) and || (Boolean or), @Pointcut, Named pointcuts, Advice, @Around, Joinpoint , steps of AOP Working

Exception handling

@ExceptionHandler, @ControllerAdvice, @GlobalExceptionHandler, @ResponseStatus